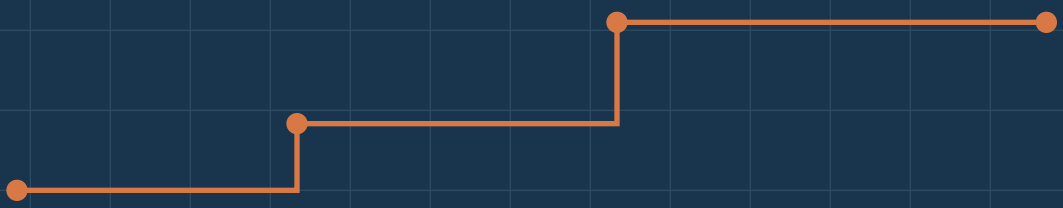
RESEARCH / COMPUTING / EVIDENCE

计算机科研入门

从第一行代码到第一篇论文

计算机科研方法与实验实践教材



01

标准化与优化

NumPy / 对照实验

02

图像分类与噪声

PyTorch / 消融分析

03

探索与安全

强化学习 / 评价协议

科研方法与技术栈 · 三个完整项目 · 可复现代码与原始记录

TEACHING EDITION / 2026.10

给第一次做研究的你

本书把科研训练拆成可以执行和检查的小步骤。你将从普通电脑出发，逐步完成环境配置、文献阅读、实验设计、模型训练、结果分析和论文写作。

阅读版与配套材料

PDF 用于连贯阅读；Markdown 源稿用于批注和修改；配套包提供三个完整程序、验证脚本、模板与本书的真实运行记录。

本书的承诺

所有标为“本书实测”的表格与曲线均从保存的逐种子记录生成。示意图、空白模板、待验证扩展和贯通测试都有明确标记。失败种子被保留，研究边界会与结果一起解释。

使用约定

2026 年 10 月教学版。代码和文本为教学用途编写，数据与参考资料的权利归原作者。任何学术投稿都需要另外核对创新性和当年规定。

> 问题可以很小，证据要足够清楚。

目录

00 如何使用这本书 5

01 科研的基本单位是证据 7

02 把兴趣变成一个能完成的课题 9

03 本地工作台：文件、解释器与环境 11

04 研究够用的 Python 与 NumPy 13

05 命令行、Linux 与进程 15

06 Git：给实验建立时间坐标 17

07 服务器、SSH、GPU 与调度 19

08 为实验补数学：先会读，再会推 21

09 检索、读论文与文献笔记 23

10 实验设计：把比较变公平 25

11 调试、记录与可复现性 27

12 项目一：标准化究竟改变了什么 29

13 项目一：实现、运行与验收 31

14 项目一：从结果到一篇短报告 33

15 项目二：进入深度学习实验 35

16 项目二：PyTorch 训练与检查点 37

17 项目二：鲁棒性、消融与短论文 39

18 项目三：强化学习的问题设定 42

19 项目三：实现 Q-learning 与 SARSA 44

20 项目三：从失败中形成研究结论 46

21 统计与作图：让数字经得起追问 49

22 把证据组织成论文 51

23 复现包、同行反馈与投稿 53

24 继续走向独立研究 55

附录 A 可直接使用的研究模板 56

附录 B 排错速查与练习答案 58

附录 C 配套包与本次运行说明 60

附录 D 首次实验：环境准备与运行 62

参考资料与继续阅读 64

FOUNDATIONS

00 如何使用这本书

你要学会什么

本书面向希望从零开展计算机科研与实验的读者，不要求已有服务器账号或独立显卡。读完本书的目标是：能独立把一个小问题变成实验，解释结果，整理成一篇结构完整的研究报告，并把它交给他人复现。你不必先学完所有数学和编程知识；遇到一个概念，就补足当前任务需要的那一部分。

本书的主线是**问题 → 协议 → 实现 → 验证 → 测量 → 论证 → 交流**。工具总是服务于这条主线。你可以很熟练地使用终端，却仍做出无效比较；也可以先用普通笔记本完成一个严谨的小研究。

如果还没有安装过 Python 或运行过脚本，附录 D 提供环境准备与首次实验的具体操作。

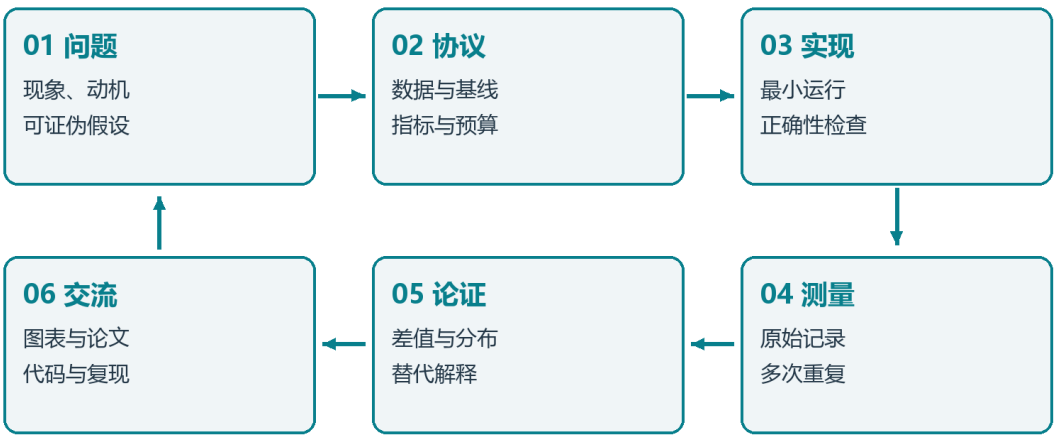


图 0-1：从问题到论文的证据链

三个项目是一条训练阶梯

项目	研究问题	核心新能力	计算条件	最终交付
01 标准化与优化	不同尺度的输入怎样影响有限步训练?	数据划分、验证集调参、配对比较	CPU；NumPy	4 页实验报告
02 图像分类与噪声	模型结构与训练噪声如何改变鲁棒性?	PyTorch、消融、检查点、预算记录	CPU 可完成；GPU 可加速	6 页短论文
03 探索与安全	为什么在线表现和最终贪心策略会给出不同排名?	MDP、时序差分、多种评估协议	CPU；NumPy	6 至 8 页研究报告

项目三的模型很小，但方法论更难：数据由策略自己产生，回报高度相关，“算法好不好”取决于你怎样评估。难度不等于模型大小。

读书时的四种标记

- **概念**：先能用自己的话解释，再往下读。
- **动手**：需要实际运行或写出产物，光看不算完成。
- **检查点**：给出完成标准，用来决定能否进入下一阶段。
- **研究边界**：说明结论能支持什么，哪些仍未验证。

代码块标注了 `python`、`powershell`、`bash`、`json` 或 `text`。一行开头的 `$` 如果出现在说明里，表示提示符，不是命令的一部分。本书正式命令块不加提示符。PDF 中代码块保持等宽；复制后仍要检查引号与缩进，完整脚本以配套包为准。

动手练习：建立一个 `research-notes.md`，写下你想研究的一个现象和可用的计算资源。写“我想了解图像噪声对小模型的影响”就足够，不需要一开始就提出新算法。

FOUNDATIONS

01 科研的基本单位是证据

从“做出东西”走向“回答问题”

课程作业常给出明确输入和标准答案；工程开发常以功能可用、延迟可接受为目标；科研还要解释：为什么值得研究？已有工作知道什么？你的结果新增了哪一条可信知识？三者可以重叠，一段好代码既能是工程成果，也能成为科研证据的工具。

例如“我训练了一个服饰分类器”说明你完成了实现。“在相同训练样本与预先声明的预算下，加入高斯噪声是否改善受损图像上的准确率，并付出多少干净准确率代价？”才是可以检验的研究问题。

一个最小研究单元由五部分组成：问题、假设、操作、观察、解释。 假设必须有失败的可能。“我的方法会有更好表现”太模糊；“在测试噪声标准差为 0.3 时，增强模型的平均准确率高于同结构对照模型”可以被数据反驳。

观察与解释分开写

观察是“本协议下模型 A 的 5 个种子平均准确率高于 B”。解释可能是“A 的局部结构先验更适合图像”。后者还需要额外证据：换结构、控制参数量、观察学习曲线等。一个差异不能自动证明你最喜欢的机制。

你还应列出替代解释：A 是否训练更久？是否使用更多样本？是否调了更多次参数？是否在测试集上偷偷挑选设置？这些问题看似苛刻，实际是在帮助你保护结论。

研究贡献有哪些形式

贡献形式	初学者可以做的版本	证据要求
复现	在声明的环境中重现一个现象	明确原设置、偏差与容差
测量	测试某个条件下的性能变化	固定协议、重复运行、误差估计
分析	找到一个常见失败条件	反例、错误分类、排除替代解释
方法	修改算法或系统组件	强基线、公平预算、消融与边界
工具或数据	让一种问题更容易研究	正确性、适用场景、可用性验证

本书三个项目首先是教学复现和小型测量研究。把教程里的实验再跑一遍，能证明学习成果，但不能直接宣称学术创新。若要投稿，需要进一步检索相关工作，找到尚未被充分回答的问题，并补充证据。

动手：写出一条证据链

选一个生活化的计算机现象，比如“训练有时变慢”。依次写：可能原因、可测量量、控制条件、可能推翻原因的结果。例：若怀疑数据读取慢，就保持模型与批大小不变，比较真实数据读取和内存中固定张量的每步耗时。若二者同样慢，读取瓶颈解释就没有得到支持。

检查点：你能否用两句话分别说出“想知道什么”和“怎样判断回答是否成立”？如果还在罗列工具名称，先缩小问题。

FOUNDATIONS

02 把兴趣变成一个能完成的课题

选题从约束出发

“研究大模型” “研究强化学习”是方向，不是课题。一个可操作的课题需要清楚的边界与可检验的问题。约束包括数据是否可得、代码能否运行、计算资源能否承担、评价指标是否明确、你是否理解最基本的基线。

把兴趣写成一个句子：**在条件 C 下，对比方案 A 与基线 B，观察指标 M，以回答问题 Q。**比如“在 12,000 张训练图像与固定 5 轮训练下，对比 CNN 的有噪声训练和无噪声训练，观察不同测试噪声下的准确率”。句子里的每一项都应该能落实到配置文件或代码。

一页立项书

MARKDOWN

```
# 项目标题
## 问题与动机
谁会在什么条件下遇到这个问题?
## 已有认识
最接近的 3 至 5 篇工作，各回答了什么?
## 假设
改变 X 会怎样影响 Y? 什么结果会推翻预期?
## 实验协议
数据、划分、基线、指标、预算、种子、选择规则。
## 风险与止损
如果数据不可用或实现无法通过最小检查，怎样缩小范围?
## 交付
代码、原始记录、图表、报告及复现命令。
```

这份文档会随着理解更新。正式评估前，将关键协议冻结并提交 Git；后来的探索另开版本。冻结不是禁止思考，而是让别人知道哪些决定来自先验计划，哪些受到结果启发。

最小可行实验

最小可行实验是能验证核心管线的一次最小运行：数据可读、模型可训练、指标可计算、记录可保存。它通常只用很少样本和一两个批次。它验证“程序贯通”，不验证“研究结论成立”。

设计三个停止条件。第一，代码不能通过最小检查时停止扩大规模。第二，预算耗尽时保存已有结果，说明未完成部分。第三，发现数据泄漏时暂停解释性能，修复协议并标记受影响的旧结果。不要把无法解释的数字堆成更大的表。

与导师或学长讨论什么

带着一页立项书和一个具体问题去讨论，比只说“我想科研”更有效。可以问：“这个问题是否已经被某篇工作回答？”“目前比较漏掉了哪个基线？”“如果只能补一个实验，哪个最能改变结论？”

动手：把“提高强化学习效果”改写为三种不同课题，分别围绕样本效率、探索风险、计算时间。为每个课题写一个指标。你会发现，同一个方向可以产生完全不同的研究协议。

FOUNDATIONS

03 本地工作台：文件、解释器与环境

电脑里发生了什么

编辑器负责改文件，终端负责发命令，Python 解释器执行程序，虚拟环境决定程序可以导入哪些包。Jupyter Notebook 适合探索和画图，命令行脚本更适合批量、远程和可复现运行。它们是互补工具。

先准备一个普通用户可写的项目目录，不要把所有东西堆在桌面。以下结构适用于三个项目：

TEXT

```
research-starter/
  README.md              # 怎样安装和运行
  protocol.md            # 当前实验协议
  p1_scaling.py          # 项目一入口
  p2_fashion.py          # 项目二入口
  p3_cliff.py            # 项目三入口
  common.py              # 元数据与保存
  analyze.py             # 按种子汇总
  data/                  # 原始数据，通常不进 Git
  runs/                  # 每次运行单独一个目录
  notes/                 # 文献卡、日志和草稿
```

在 Windows 上建立环境

建议使用 Python 3.12 作为本书学习环境，并为项目建立独立虚拟环境。以下命令在解压后的代码目录执行。使用虚拟环境解释器的完整相对路径，可以避开 PowerShell 激活脚本的策略问题。[1]

POWERSHELL

```
python --version
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements-core.txt
.\.venv\Scripts\python.exe p1_scaling.py --out runs/p1-dev
```

Linux 或 macOS 对应写法：

BASH

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements-core.txt
.venv/bin/python p1_scaling.py --out runs/p1-dev
```

`-m pip` 的含义是“让这个 Python 运行 pip 模块”。它有助于避免 `pip` 和 `python` 指向不同环境。确认环境时，运行 `python -c "import sys; print(sys.executable)"`，看路径是否属于你准备使用的环境。

深度学习依赖

项目一和项目三只需 NumPy。项目二另外需要 PyTorch，不依赖 torchvision。先按官方安装选择器选择操作系统和 CPU/CUDA 平台，再安装与记录实际版本。[2] CPU 版的学习环境可以这样准备：

```powershell
.\.venv\Scripts\python.exe -m pip install numpy
.\.venv\Scripts\python.exe -m pip install torch --index-url
    https://download.pytorch.org/whl/cpu
.\.venv\Scripts\python.exe -c "import torch; print(torch.__version__)"
.\.venv\Scripts\python.exe -m pip freeze > requirements-lock.txt
```

包内的 `requirements-core.txt` 与 `requirements-dl.txt` 给出兼容范围，便于首次安装；它们不是跨机器精确锁文件。正式运行后保存 `pip freeze`、Python 版本、操作系统和设备信息。GPU 环境还要保存驱动、PyTorch 的 CUDA 构建版本及 GPU 型号。

三类常见故障

现象	先检查什么	一个具体动作
`ModuleNotFoundError`	包是否装在当前解释器	用同一个解释器运行 `-m pip show numpy`
找不到文件	当前目录与路径基准	打印 `Path.cwd()` 与目标绝对路径
下载失败	网络、代理、证书与文件完整性	使用可信来源重下并检查校验值

检查点：关闭编辑器后，你仍能从新终端运行项目一，并找到它打印的输出目录。成功一次不够；你要知道这次成功用了哪个解释器。

FOUNDATIONS

04 研究够用的 Python 与 NumPy

从一条数据管线开始学

不必在入门阶段背完整语法。先掌握变量、条件、循环、函数、列表、字典、文件和异常，然后用它们完成“加载 → 检查 → 计算 → 保存”。研究中最危险的错误常不是语法错，而是程序顺利执行却算错了维度或样本。

PYTHON

```
from pathlib import Path
import json
import numpy as np

rng = np.random.default_rng(0)
x = rng.normal(size=(8, 3))
y = (x[:, 0] > 0).astype(int)
assert x.shape == (8, 3)
assert len(x) == len(y)
record = {"n": len(y), "positive_rate": float(y.mean())}
Path("summary.json").write_text(
    json.dumps(record, indent=2), encoding="utf-8"
)
```

`x[:, 0]` 表示所有样本的第一个特征。`axis=0` 常表示沿样本方向聚合，得到每个特征的统计量。NumPy 的广播能省代码，也能悄悄制造错误：形状 `(N, 1)` 减去 `(N,)` 会得到 `(N, N)`，而不是你期待的逐样本差值。每次重要运算都先写出形状。

用函数缩小理解范围

一个函数应完成一件可说明的事，例如“根据训练集计算均值和标准差”，或“在不更新参数时评估一个模型”。函数输入输出越清楚，实验越容易替换组件。全局变量越多，越难判断一次改动到底影响哪里。

推荐为关键函数写短文档：输入形状、返回形状、是否修改输入、是否使用随机数。不要让 `evaluate()` 默默做训练，也不要让 `load_data()` 默默重划分数据。

错误信息怎么读

从 traceback 最后一行读起：错误类型和直接原因是什么？再向上找第一处属于自己项目的代码。`ValueError` 常是输入不符合预期，`IndexError` 常是索引越界，`RuntimeError` 在深度学习中常涉及形状、设备或资源。

最小复现是调试的核心：把导致错误的代码缩到十几行，固定输入，打印形状和 dtype。贴出完整几千行日志通常比不上提供一个能立即复现问题的小程序。

动手与答案提示

1. 生成形状 `(100, 5)` 的数组，计算每列均值，再标准化每列。
2. 让第三列全为常数，观察除以标准差会怎样。用 `np.maximum(sd, 1e-8)` 避免除零，并说明常数列不会因此获得信息。
3. 把随机种子换成 1，确认数值变化；重新用 0，确认同一环境下数值恢复。

检查点：你能说明 `(x - mu) / sd` 中三个数组的形状，并能解释为什么结果仍是 `(100, 5)`。

FOUNDATIONS

05 命令行、Linux 与进程

先辨认自己在哪台机器

终端中最重要的三个问题是：我是谁？我在哪？现在运行的是谁的程序？远程登录后，你输入的命令通常在服务器执行，文件路径也属于服务器。本地 Windows 的盘符路径不会自动出现在 Linux 上。

下面是 Linux Bash 命令；不要直接把它们当作 PowerShell 命令使用。更系统的工具练习可参考 MIT 的 Missing Semester 课程。[3]

```bash
whoami
hostname
pwd
ls -lh
mkdir -p runs notes
python -c "import sys; print(sys.executable)"
df -h .
du -sh data runs
```

`df` 看文件系统余量，`du` 看目录占用；它们回答不同问题。进程是正在运行的程序实例。终端关闭可能影响绑定在终端上的进程，因此长训练不能只依赖一个裸露的 SSH 会话。

标准输出、错误与日志

程序的普通输出和错误输出是两条流。下面的 Bash 写法把二者一起保存，并在屏幕显示：

```bash
set -o pipefail
python p1_scaling.py --out runs/p1-log 2>&1 | tee p1.log
```

`pipefail` 让管线中前面程序的失败不被 `tee` 成功掩盖。正式实验除了日志，还应保存结构化 JSON/CSV。日志适合排错，结构化文件适合统计；从屏幕抄数字容易漏项。

长运行与资源观察

在允许交互任务的机器上，可使用已安装的 `tmux`：`tmux new -s study` 建立会话；按 `Ctrl+B`，松开后按 `D` 分离；`tmux attach -t study` 返回。学校集群若要求调度器，必须按集群规则提交任务，不能以 `tmux` 代替资源申请。

查看 CPU 可用 `top`，查看当前用户进程可用 `ps -u "$USER"`。结束程序前先确认进程归属与输出保存情况，优先在程序终端按 `Ctrl+C`。不要把强制终止当作正常保存方法。

动手：做一次可靠运行

在一个新输出目录运行项目一，保存日志；故意提供一个不存在的脚本路径，观察退出信息；再故意把输出目录复用，确认程序拒绝覆盖。科研目录里“失败了但保留证据”往往比“看似成功但覆盖历史”更有价值。

FOUNDATIONS

06 Git：给实验建立时间坐标

代码版本与实验版本

Git 保存版本快照，便于回到过去、比较差异和协作。[4] 研究记录需要把一次运行与某个代码版本关联。仅记录“用了最新版”没有意义，因为最新版会变化。

```bash
git init
git status
git add README.md protocol.md p1_scaling.py common.py .gitignore
git commit -m "Add first controlled experiment"
git log --oneline -5
git diff
git rev-parse HEAD
```

首次提交可能需要按提示配置姓名和邮箱。`git add` 把选定变更放入暂存区，`commit` 为这一批变更建立快照。`diff` 用于检查尚未提交的变化。先看差异，再提交，是一个很实用的习惯。

什么应该进入仓库

代码、小配置、协议、说明和小型结果汇总适合版本管理。大数据、模型权重、虚拟环境、密钥和临时缓存通常不适合直接进入普通 Git 仓库。`.gitignore` 只影响未被追踪的文件；已提交的敏感内容不会因为后来添加忽略规则就消失。

配套代码会尽力保存 commit 与 dirty 状态。`dirty=true` 表示工作目录存在未提交改动；此时单独一个 commit 无法完整重现当前代码。正式运行前要么提交相关变更，要么保存补丁与实际源文件副本。

一次运行的最小档案

```json
{
  "question": "Does scaling help under 400 updates?",
  "commit": "从 git rev-parse HEAD 获取",
  "config": {"steps": 400, "seeds": [0, 1, 2, 3, 4]},
  "selection_rule": "minimum validation loss",
  "status": "planned"
}
```

这里是字段示范，不是实际实验记录。配套程序会生成真实 `metadata.json`，保存完整配置、时间、Python/NumPy 版本和配置哈希。配置哈希只能帮助识别配置是否相同，不证明代码或数据相同。

检查点：随机抽取一个结果目录，你能找到它对应的命令、配置和代码吗？如果不能，先完善档案，再增加新实验。

FOUNDATIONS

07 服务器、SSH、GPU 与调度

什么时候需要服务器

当本地机器太慢、内存不足，或实验必须在指定硬件上运行时，再使用服务器。项目一和三在普通 CPU 上即可完成；项目二也能用 CPU。先在小数据上验证，再把已知能跑通的程序迁移过去。

服务器账号由学校或课题组提供。下面的域名和用户名是示例，需要替换成真实信息。首次连接应向管理员核对主机指纹。[5]

```bash
ssh student@gpu.example.edu
```

本地与远端传文件可使用 `scp`。在本地终端执行：

```bash
scp p1_scaling.py common.py student@gpu.example.edu:~/research/
scp -r student@gpu.example.edu:~/research/runs/p1-main ./downloaded-p1
```

需要跳板机时用管理员提供的跳转方式，例如 `ssh -J student@login.example.edu student@gpu.example.edu`。私钥属于你自己；公钥才交给管理员登记。

GPU 三层关系

驱动让操作系统使用 GPU；**PyTorch 的 GPU 构建**提供相应运行时支持；**项目代码**决定把张量与模型放在哪个设备。`nvidia-smi` 显示的 CUDA 版本是驱动支持能力的提示，不等于项目安装了同版本 CUDA toolkit，也不保证当前 PyTorch 能使用 GPU。

```bash
nvidia-smi
python -c "import torch; print(torch.__version__)"
python -c "import torch; print(torch.version.cuda)"
python -c "import torch; print(torch.cuda.is_available())"
```

若 CUDA 可用，再把项目二命令加上 `--device cuda`。显存不足时先减小批大小。不要先升级服务器驱动；共享机器的软件栈由管理员管理。

Slurm 最小模板

以下模板假设集群安装 Slurm，且允许 `--gres=gpu:1`。分区名、账号、资源请求和环境路径以本校说明为准。[6]

```BASH
#!/bin/bash
#SBATCH --job-name=fashion-study
#SBATCH --gres=gpu:1
#SBATCH --cpus-per-task=2
#SBATCH --mem=8G
#SBATCH --time=00:30:00
#SBATCH --output=logs/%j.out
set -euo pipefail
cd "$SLURM_SUBMIT_DIR"
.venv/bin/python p2_fashion.py --device cuda --out runs/p2-gpu-dev
```

提交前创建 `logs/`，提前准备数据和 Python 环境。运行 `sbatch job.sh` 得到作业号，`squeue -u "$USER"` 看队列，用 `sacct -j` 查看结束状态。排队不是程序卡住；提交成功也不表示训练已开始。

远程 Notebook

需要 Notebook 时让服务只监听服务器回环地址，再通过 SSH 转发访问。例如远端运行 `jupyter lab --no-browser --ip=127.0.0.1 --port=8888`；本地另开终端运行 `ssh -N -L 127.0.0.1:8888:127.0.0.1:8888 student@gpu.example.edu`。本地浏览器使用服务给出的带 token 地址。不要把带访问 token 的链接提交到仓库。

检查点：能从日志区分“环境未准备好”“资源尚未分配”“程序已启动但失败”三种情况。

FOUNDATIONS

08 为实验补数学：先会读，再会推

线性代数是一套形状规则

把一个批次写成矩阵 X，每行是一个样本，每列是一个特征。若批次含 N 个样本、每个样本有 d 个特征，输入、权重和输出分数的形状分别为：

$$X \in \mathbb{R}^{N \times d}, \qquad w \in \mathbb{R}^{d}, \qquad Xw \in \mathbb{R}^{N}.$$

很多深度学习错误都能靠写下这三个形状发现。

向量点积表示加权求和；矩阵乘法把许多点积并行完成。范数衡量大小，特征值和条件数帮助理解优化的几何。本书项目一只需要先理解：不同方向的尺度差距太大，可能让梯度下降在某些方向受步长限制。

微积分告诉你怎样改变参数

损失函数把当前预测的坏程度变成一个数。梯度给出各参数局部变化对损失的影响。梯度下降的更新为：

$$w_{t+1} = w_t - \eta \nabla_w L(w_t)$$

η 是学习率。太小可能进展缓慢，太大可能发散。偏导不是一个神秘动作：先固定其他参数，看改变这一项会使损失怎样变化。深度学习的自动微分负责计算梯度，但不会替你判断损失是否定义正确。

概率描述不确定性

二分类模型输出给定输入时属于正类的条件概率：

$$p(y = 1 \mid x).$$

概率为 0.8 不表示每次都必然正确，而是希望在一类可比较情形中，预测为 0.8 的样本有约 80% 为正。准确率描述分类决策，交叉熵还惩罚过度自信的错误，两者回答不同问题。

期望是长期平均；方差描述离散程度。实验里的随机性包括采样、数据划分、参数初始化、批次顺序和环境交互。固定一个随机种子有助调试，不能替代多次独立运行。[7]

统计量的单位必须明确

同一训练模型在 10,000 个测试样本上的表现，不等于 10,000 次独立训练。多个 epoch 的指标同样高度相关，不能把 5 个种子各 10 个 epoch 当成 50 次独立实验。

若有 n 个独立种子，每个种子得到一个分数，样本均值为：

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i.$$

当 n 大于 1 时，样本标准差为：

$$s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2}.$$

均值 ± 标准差描述这些运行的离散，不是均值的 95% 置信区间。本书后面用按种子重采样的 bootstrap 演示区间估计，同时保留每个种子的原始值。

动手

对分数 `[0.80, 0.82, 0.78, 0.81, 0.79]` 手算平均值，再用 `np.std(values, ddof=1)` 算样本标准差。平均值为 0.80，样本标准差约 0.0158。它们都以“准确率比例”为单位；若换成百分数，二者都乘 100。

FOUNDATIONS

09 检索、读论文与文献笔记

检索不是只找支持自己的文章

从问题拆关键词：任务、方法、条件、评价。项目二可以组合 `image classification`、`Gaussian noise augmentation`、`robustness`、`Fashion-MNIST`。先找到综述或经典定义，再向前追参考文献，向后查引用它的研究。检索结果越符合预期，越要主动找反例和负面结果。

arXiv 预印本、正式发表版本、作者代码仓库承担不同角色。记录你实际阅读的版本和日期，检查论文标题、作者、年份、链接是否互相对应。引用存在的论文，也要确保它确实支持你所写的那一句话。

三轮阅读法

Keshav 提出的三轮阅读法可概括为：第一轮判断问题与贡献，第二轮理解方法和证据，第三轮深入推导或尝试复现。[8] 不需要每篇文章都读到第三轮。

第一轮阅读提取问题、核心主张、主要图表和一项不确定点。第二轮带着协议问题阅读：数据怎么划分？最强基线是什么？参数怎么选？误差条表示什么？第三轮逐段核对公式和实现。

一张论文卡

MARKDOWN

```
# 文献卡：作者 年份 短标题
- 链接与版本：
- 一句话问题：
- 作者主张：
- 最关键的图或表：
- 我理解的方法：
- 选择模型与评估协议：
- 局限与尚未验证的条件：
- 与我课题的关系：支持 / 反例 / 基线 / 工具
- 下一步：复现哪一个最小现象？
```

读不懂时要区分三类阻塞。术语阻塞查定义；公式阻塞先找变量形状与目标；论证阻塞问“哪一条证据支撑哪一句主张”。先补课再回来，比对着陌生符号反复通读更有效。

做一个相关工作矩阵

每行一篇论文，列包括任务、数据、方法、预算、指标、适用条件、局限。矩阵的作用是暴露空白和比较差异，而不是拼出“所有前人都不行”的故事。基线如果出自某篇论文，应准确注明；自己改了数据或预算，就说明这已经是新的评估设置。

检查点：交给同学 5 张论文卡，他们能否明白这些工作之间是什么关系，以及你的问题与它们相差在哪里？

FOUNDATIONS

10 实验设计：把比较变公平

先写清楚实验的角色

训练集用于学习参数；验证集用于选学习率、结构、检查点等；测试集用于协议冻结后的最终估计。把三者混在一起，会让你难以判断模型是在学习任务，还是在适应反复观察过的数据。

数据处理也可能学习参数。标准化的均值与标准差、PCA 的主轴、特征筛选规则，都应仅用训练数据拟合，再应用于验证和测试。[9] 对时间序列不能随意随机打乱；对同一人的多张图像，按图像随机分割可能让同一身份跨集合。划分单位要与实际泛化目标一致。



图 10-1：训练、验证、测试各自的职责

公平并非只有一种定义

固定训练轮数比较模型时，大模型可能用更多计算。固定墙钟时间比较时，不同设备和实现又会影响结果。你可以选择任何与问题一致的预算，但必须声明，并补充关键成本。至少记录样本数、更新步数、参数量、训练耗时；GPU 实验再记录设备型号与峰值显存。

调参次数也是预算。方案 A 试 100 个配置、基线只试 1 个，很容易得出误导性的结论。最实用的做法是预先给出相同候选数量，并按同一验证指标选择。相同候选集也未必覆盖每个方法的最佳区间，论文中要承认这一点。

基线与消融

基线是你必须超过或解释清楚的参照，包括简单方法、标准方法以及与你方案最接近的方法。消融是把方案中的一个组件移除或替换，以检查它的作用。

项目二的三个组是 MLP、CNN、CNN 加噪声。MLP 与 CNN 比较包含结构和参数量等差异，不能把所有差异归因于卷积；CNN 与 CNN 加噪声只改变增强设置，更适合解释增强的条件效果。

协议冻结与测试集使用

在开发阶段运行不带 `--finalize` 的项目一或二，只输出验证结果。写清候选集、种子和选择规则后，再加 `--finalize` 获得测试结果。这个参数是操作提醒，不是技术隔离；科研诚实仍依赖你遵守协议。

如果看过测试结果后又改配置，应把后续工作标为探索，说明测试集已被反复观察。要形成新的确认性结论，需要新的未见评估数据或明确的外部验证方案；换个输出目录不会让旧测试集重新变“未见”。

检查点：在运行前，你能写出一个不依赖最终结果的规则，说明究竟选哪个模型吗？

FOUNDATIONS

11 调试、记录与可复现性

先证明程序会做正确的事

深度学习的实用调试顺序是：检查数据和标签 → 跑通前向传播 → 检查损失与梯度 → 过拟合一个小批次 → 验证评估模式 → 扩大数据。小批次过拟合失败时，不要立即启动大规模训练。

对分类任务检查标签是否在合法范围，输入是否缩放正确，训练与验证是否重叠。对强化学习检查环境每一步的状态、奖励、终止条件。看十条真实转移，常比盲目调参更快找到问题。

种子与确定性

固定 Python、NumPy、PyTorch 的随机源，必要时开启确定性算法，并记录失败或性能代价。PyTorch 官方说明，不同版本、平台或设备之间不能保证完全相同的结果。[7] 因此复现应声明容差与环境，不能只写“固定 seed 即可”。

数据加载的多进程 worker 也可能引入随机性。入门阶段先使用 `num_workers=0`，等逻辑正确后再优化吞吐。项目二正是采用这个简化设置。

一次运行保存什么

类别	至少保存	为什么
身份	run ID、UTC 时间、代码版本	让别人定位运行
协议	完整命令、参数、种子、划分索引	说明测量条件
环境	Python、依赖、系统、设备	解释跨机器差异
过程	训练与验证曲线、耗时、错误日志	判断训练是否正常
结果	每种子指标、预测、检查点	支持重算与错误分析
决策	选择规则、协议修改与原因	避免事后挑选

配套代码自动保存其中大部分；完整软件锁文件、GPU 型号和研究笔记仍由你补齐。程序保存了数值，不等于自动完成了研究记录。

最小正确性检查

- 数据索引两两无交集，且所有索引都合法。
- 一个已知输入能得到手算的输出或奖励。
- 评估前后参数保持相同，梯度不累积。
- 同一输入、同一环境和种子能够重现调试过程。
- 输出不能静默覆盖前一次运行，失败不会伪装成成功。

动手：在项目三里手算从起点向右一步的结果，再运行 `transition(36, 1)` 核对。正确返回应包含起点 36、奖励 -100、未终止和一次跌落。这一条检查可以直接发现“把跌落当终止”的环境实现错误。

PROJECT LAB

12 项目一：标准化究竟改变了什么

项目卡与立项

问题：在特征尺度差异明显的二分类数据上，用相同数量的梯度更新和相同候选数量调参，标准化能否改善有限预算下的分类性能？

假设：标准化后，不同特征方向的梯度尺度更接近，较大的稳定学习率更容易发挥作用，因此在 400 次全批量更新内达到更低的验证损失。反例可能是原始输入经充分调参和更长训练后同样有效。

研究定位：这是一个可控制数据生成机制的优化教学实验。它不衡量某个真实行业任务，也不主张标准化总能提升模型的统计上限。

数据是怎样生成的

先生成 2,000 个 12 维标准正态隐变量 z。前六维参与真实标签概率，其余六维与标签机制无关。标签按伯努利分布随机采样；因此即使知道真实概率，也不保证所有样本都能正确分类。

接着让各输入特征乘上从 0.1 到 100 的几何尺度。尺度最大的几个维度恰好是无关特征，故意制造对原始梯度下降不友好的情形。**这是有意设计的压力测试，不能代表所有数据分布。**

PYTHON

```
z = rng.normal(size=(2000, 12))
beta = np.array([2, -2, 1.5, -1.5, 1, -1] + [0] * 6)
y = (rng.random(2000) < sigmoid(z @ beta)).astype(float)
x = z * np.geomspace(0.1, 100, 12)
```

每个种子重新生成数据，再随机划成 1,200 条训练、400 条验证、400 条测试。两组方法在同一个种子内使用完全相同的数据与索引，因此适合做配对比较。跨种子变化包含数据采样和划分变化；不是只测初始化波动。

模型和目标函数

逻辑回归先计算线性分数，再通过 sigmoid 得到正类概率：

$$p_i = \sigma(x_i^\top w + b), \qquad \sigma(a) = \frac{1}{1 + e^{-a}}$$

训练使用二元交叉熵。实现中用 `np.logaddexp` 稳定地计算损失，避免直接对接近 0 的概率取对数。

$$L = \frac{1}{N}\sum_{i=1}^{N}\left[\log(1+e^{a_i}) - y_i a_i\right]$$

主实验不使用 L2 正则化，因为标准化还会改变“同一数值正则系数”在原始坐标中的含义。先移除这个混杂因素，才能更清楚地讨论有限步优化。偏置通过在输入末尾添加一列 1 实现。

冻结协议

项目	预先声明的设置
方法	原始输入；仅用训练集拟合标准化
初始化与优化	全零权重；全批量梯度下降；400 步
学习率候选	0.0001、0.001、0.01、0.1
选择规则	每组每种子取验证交叉熵最低者；平局取较小学习率
最终指标	测试准确率与交叉熵；分类阈值固定为 0.5
重复	种子 0、1、2、3、4；保留所有种子

同一候选集并不保证两组得到同等充分的搜索。它定义的是“这个搜索预算下的效果”。如果想讨论各方法充分优化后的能力，必须扩大原始组的候选范围和训练预算，并另立协议。

动手：先不运行程序，画出你预期的两条训练损失曲线，并写一句可能推翻假设的观察。之后把草图和实际曲线一起保留。

PROJECT LAB

13 项目一：实现、运行与验收

第一步，先划分再处理

配套 `dataset()` 返回原始输入、标签和三个索引数组。`design()` 只用训练索引计算变换参数，再把同一变换应用到全部输入。

```python
mu = x[train_ids].mean(0)
sd = np.maximum(x[train_ids].std(0), 1e-8)
x_scaled = (x - mu) / sd
```

这里“把变换应用到测试输入”本身并非泄漏；关键是变换参数没有从测试数据学习。你也可以分别调用变换函数处理三个集合，逻辑完全相同。

第二步，理解训练循环

```python
w = np.zeros(x.shape[1])
for step in range(steps):
    probabilities = sigmoid(x @ w)
    gradient = x.T @ (probabilities - y) / len(y)
    w -= lr * gradient
```

每一行都有数学含义：`x @ w` 是分数，概率减标签是误差方向，左乘 `x.T` 把误差传回特征权重。`sigmoid()` 内对指数参数做范围限制，避免浮点溢出；它保护数值计算，不会修复一个过大的学习率。

练习：在很小数据上用有限差分验证一个权重的梯度。设步长 h=0.00001，给某一参数分别加 h 和减 h，用两次损失的差除以 2h。若误差明显，先检查平均、转置和偏置处理。

第三步，开发运行

以下统一以已配置好环境的 `python` 表示当前环境解释器；Windows 可以替换为 `.\.venv\Scripts\python.exe`。

```bash
python p1_scaling.py --seeds 0 --steps 40 --out runs/p1-check
```

输出应包含两组验证损失与所选学习率，目录包含 `metadata.json`、`results.json`、划分索引和模型参数。此时没有测试指标，因为没有解锁最终评估。

检查验证集是否参与梯度计算？检查两组是否用了相同索引？检查每个候选是否都从零初始化？检查选择规则是否严格按验证损失，而非看测试准确率？

第四步，最终评估与汇总

写好 `protocol.md` 后执行：

```bash
python p1_scaling.py --finalize --out runs/p1-main
python analyze.py runs/p1-main/results.json --metric test_acc --pair standardized raw
```

`analyze.py` 会输出每组均值、样本标准差、每种子原始值，以及标准化组减原始组的配对差。配对 bootstrap 每次有放回抽取种子差值，重复 10,000 次，用 2.5% 和 97.5% 分位数演示区间估计。它没有把 400 条测试样本误当作 400 次训练。

只有 5 个种子时区间可能很不稳定；这个区间只条件于当前数据机制、搜索规则和预算。项目一同时改变数据样本与划分，所以不能把它解释成固定数据集上的纯训练随机性。

验收与交付

- 一页立项书，包含压力测试性质和假设。
- 两组各 5 条最终结果，所有学习率候选的验证损失。
- 保存的划分、模型、预测与混淆矩阵。
- 两张图：训练曲线；每种子的配对准确率。
- 一段不超过 150 字的结论，明确有限预算与合成数据边界。

PROJECT LAB

14 项目一：从结果到一篇短报告

本书实测

下列数字来自本次配套程序的真实运行，不是预设目标值。环境为 Windows 11、Python 3.12.14、NumPy 2.3.5，完整运行记录和源码校验信息保存在代码包中。不同硬件或依赖版本的数值可能略有变化。

方法	测试准确率 / %	样本标准差 / 百分点	种子数
原始输入	51.60	2.48	5
训练集标准化	86.75	2.06	5

配对平均差为 **35.15 个百分点**；按数据种子重采样的示范性 95% percentile bootstrap 区间约为 **[34.00, 36.45] 个百分点**。5 个种子的区间只作方法演示，不能推广到其他数据分布。

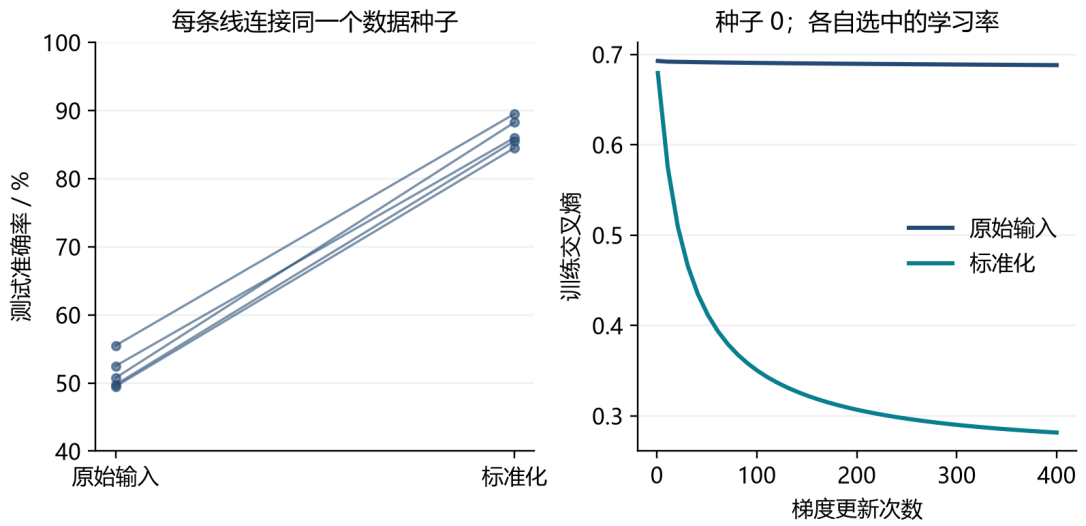


图 14-1：项目一的配对种子结果与优化轨迹

原始组最终都选择了 0.0001，标准化组都选择了 0.1。这说明在当前候选集和 400 步约束下，标准化组能够使用更有效的步长。它不证明原始输入“不能学会”，也不证明标准化创造了新的标签信息。

怎样进行错误分析

混淆矩阵的行是真实类别，列是预测类别。查看假阳性与假阴性是否明显不平衡。合成数据还有一个优势：你知道真实概率机制，可以进一步检查模型在哪些潜在概率区间错误最多。但如果使用真实机制分析结果，要注明这只在合成实验中可得。

不要只说“提升很大”。试着解释：原始组几乎停留在接近随机分类的水平，标准化组明显更高；结合训练损失轨迹，更符合“有限步优化受尺度限制”的解释。但进一步证明机制，仍需补充尺度比、步数和学习率范围的系统实验。

报告结构示范

题目：异尺度合成分类数据上标准化对有限步逻辑回归优化的影响。

摘要骨架：我们研究〔条件〕下的〔问题〕。在〔数据生成与预算〕中，对比〔两组方法〕，并通过〔验证规则〕选择配置。在〔种子数量〕次重复中观察到〔已计算数字〕。这些结果支持〔有限范围结论〕，但不涉及〔尚未验证范围〕。

正文用四页完成：第一页引言和问题；第二页数据、损失与协议；第三页结果图表和误差分析；第四页讨论、局限、复现说明与引用。页数是练习目标，不是某个会议的格式要求。

继续研究的三个方向

1. 改变最大与最小尺度比，观察标准化优势何时开始出现。
2. 增加原始组搜索范围和步数，区分“更快优化”与“更高上限”。
3. 加入 L2 正则化，分别讨论原始坐标与标准化坐标中的惩罚含义。

这些是扩展研究提议，书中主实验没有验证。既有测试结果已被你阅读，扩展应标为探索，并保存新的协议版本。

PROJECT LAB

15 项目二：进入深度学习实验

从经典模型到神经网络

神经网络把许多带参数的变换组合起来。MLP 把图像展平成向量，再通过全连接层学习表示；CNN 通过局部卷积、参数共享和池化利用图像结构。反向传播使用链式法则计算梯度，优化器根据梯度更新参数。系统学习可以参考 Goodfellow 等人的《Deep Learning》。[10]

现在你要问的不是“CNN 能不能识别服饰”，而是：**在固定训练样本、轮数和优化设置下，结构与噪声增强如何改变干净与受噪图像上的表现？**

数据与任务

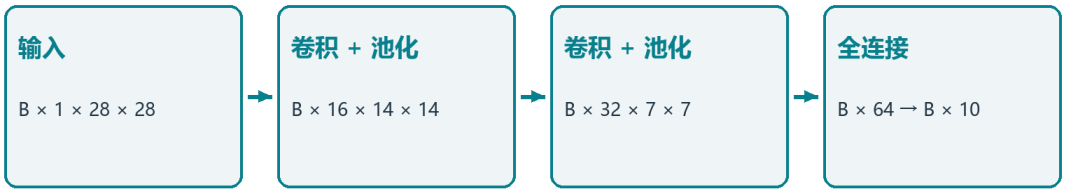
Fashion-MNIST 包含 28×28 灰度服饰图像，10 个类别，官方划分为 60,000 条训练和 10,000 条测试。[11] 本书在官方训练集内固定随机种子 2026，抽取 12,000 条训练和 3,000 条验证，其余官方训练样本不使用。官方测试集全部保留到协议冻结后的评估。

脚本直接读取官方 IDX 压缩文件，不依赖 torchvision。首次使用 `--download` 下载，并核对每个文件的 MD5。这里校验用于发现下载损坏和意外文件变化，不将 MD5 当作对恶意来源的安全保证。保留数据来源与数据许可证说明。

三组方案

组名	结构与处理	对照意义
`mlp`	展平 → 128 单元 → ReLU → 10 类	简单神经网络基线
`cnn`	两次卷积与池化 → 64 单元 → 10 类	图像结构基线
`cnn_noise`	与 CNN 完全同构；训练输入加入噪声	隔离训练噪声的条件作用

MLP 有 101,770 个参数，CNN 两组均有 105,866 个参数，规模相近但不完全一致。即使参数数目完全相同，也不能证明计算成本相同；卷积的运算模式、激活大小和优化难度都不同。



输出是 10 类 logits；交叉熵输入不需要先做 softmax。

图 15-1：图像模型的形状变化

噪声协议

输入像素缩放到 `[0,1]`。增强组训练时对每个批次独立加入标准差 0.15 的高斯噪声，然后裁剪回 `[0,1]`。验证集始终保持干净，用干净验证交叉熵选择检查点。测试时分别评估噪声标准差 0、0.15、0.3。

同一种子、同一测试噪声下，各组使用相同的噪声生成种子与批大小，因此比较时受到相同扰动。不同噪声级别由同一基础随机序列缩放获得，结果相互相关，不能当作独立重复。

研究边界：这里考察的是已定义的像素级高斯噪声与裁剪操作，不是所有现实拍摄条件，也不是对抗攻击鲁棒性。

PROJECT LAB

16 项目二：PyTorch 训练与检查点

读懂一个批次

PyTorch 官方基础教程从张量、数据加载到训练和保存模型建立了完整路径。[12] 本项目每批输入形状为 `(B,1,28,28)`，标签为 `(B,)` 的整数类别，模型输出 `(B,10)` 的 logits。

`CrossEntropyLoss` 接受 logits 和整数类别标签，内部包含所需的归一化计算。不要先手动 softmax 再当作 logits 输入这个损失。训练时标签用整数类型，模型与输入必须在同一设备。

```python
model.train()
for images, labels in loader:
    images = images.to(device)
    labels = labels.to(device)
    optimizer.zero_grad(set_to_none=True)
    logits = model(images)
    loss = nn.functional.cross_entropy(logits, labels)
    loss.backward()
    optimizer.step()
```

`model.train()` 切换训练行为；`zero_grad()` 清理上一批梯度；`backward()` 计算当前梯度；`step()` 更新参数。虽然本书模型没有 Dropout 和 BatchNorm，也要养成显式切换模式的习惯。

评估与保存

```python
model.eval()
with torch.no_grad():
    logits = model(images)
    predictions = logits.argmax(dim=1)
```

评估既要切到 `eval()`，也要关闭梯度记录。按每批平均损失再简单平均，可能使最后一个小批次权重过大；脚本先累加每批损失总和，再除总样本数。

训练中每轮检查干净验证损失，严格变小时保存检查点，平局保留较早一轮。使用 `state_dict` 的张量副本，防止后续更新悄悄修改“最好模型”。最终测试前先恢复这个检查点，而不是默认使用最后一轮。

开发运行与正式运行

先做不下载真实数据的随机张量贯通检查：

```bash
python p2_fashion.py --smoke --seeds 0 --epochs 1 --out runs/p2-smoke
```

随机张量与随机标签没有真实分类意义。这个模式为了节省资源，还复用了同一小份验证与测试张量，结果文件明确标为 `random_smoke`；不得把它作为科研结果。

再做真实数据开发运行：

```bash
python p2_fashion.py --download --seeds 0 --epochs 2 --out runs/p2-dev
```

正式协议使用 5 个种子、每组 5 轮、Adam、学习率 0.001、批大小 128、CPU 两线程、固定数据划分。此处学习率是预先固定的，没有另外挑选最优学习率。训练加噪声标准差固定为 0.15。

```bash
python p2_fashion.py --download --finalize --out runs/p2-main
python analyze.py runs/p2-main/results.json --metric test_acc_noise_0.3 --pair cnn_noise cnn
```

在已准备好 CUDA 环境的机器上，加 `--device cuda`。若确定性算法报告不支持某个算子，先保留错误与环境信息，再考虑更换支持的算子或明确标注为非完全确定性设置。

如何核验代码

从保存的预测重新算准确率；检查 10,000 条测试标签长度；检查训练与验证索引无交集；检查 CNN 与增强 CNN 的参数数目相等。脚本已经保存必要数组，但你仍需主动验收。

动手：计算一次卷积和池化后的空间形状。`padding=1` 的 3×3 卷积保持 28×28；2×2 最大池化变为 14×14；第二组后变成 32×7×7，因此全连接输入为 1,568。

PROJECT LAB

17 项目二：鲁棒性、消融与短论文

本书实测

下表为 5 个训练种子的测试准确率，表示为百分数的均值 ± 样本标准差。所有模型使用同一官方测试集，种子重复描述初始化、批次顺序和噪声采样的变化；它不覆盖换数据集的不确定性。

方法	干净图像 / %	噪声 0.15 / %	噪声 0.3 / %
MLP	83.04 ± 0.58	79.44 ± 1.26	63.70 ± 2.79
CNN	84.26 ± 0.78	80.74 ± 1.42	61.84 ± 7.53
CNN + 训练噪声	82.54 ± 0.59	82.06 ± 0.41	77.36 ± 1.53

增强 CNN 相对基础 CNN，在测试噪声 0.3 时平均增加 **15.52 个百分点**，在干净测试上平均减少 **1.71 个百分点**。这两者需要一起报告。

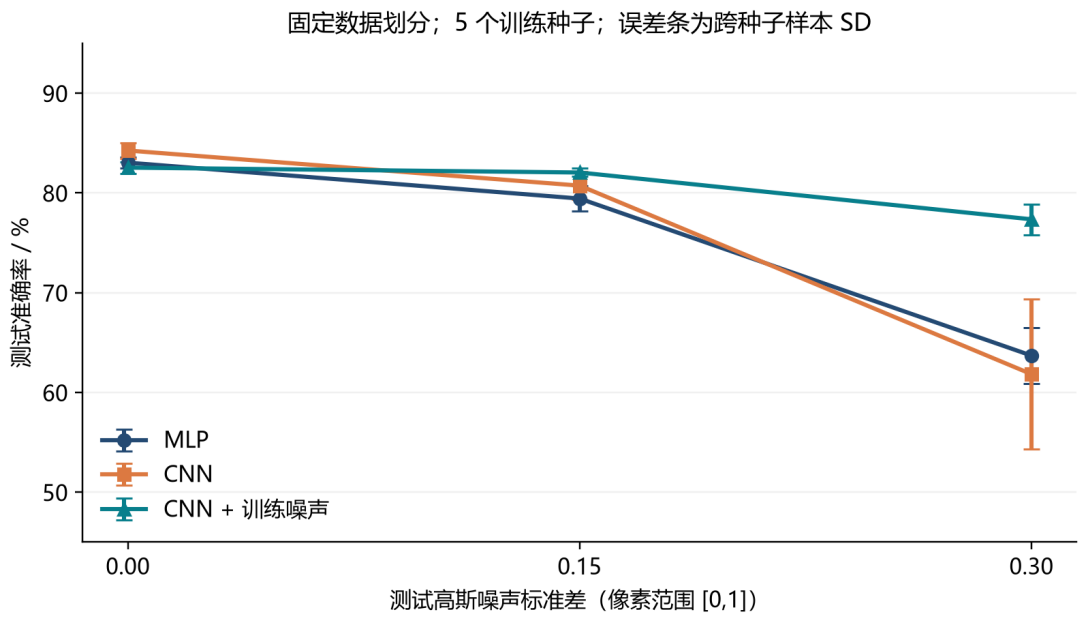


图 17-1：三个模型在不同测试噪声下的实测表现

这里的增强 CNN 在较强噪声下更稳健，同时损失了一部分干净准确率。基础 CNN 在干净图像上较好，却并没有在最强噪声下超过 MLP。不要预先认定复杂结构在所有条件都更强，也不要把训练增强的作用概括为没有代价的提升。

误差分析要从真实样本出发

先读取每种子的预测文件，按真实类别与预测类别统计错误。再回到官方测试图像，查看错误样本是否存在明显形态相近、低对比度或噪声遮挡。报告至少展示几种固定挑选规则下的样本，例如“测试索引顺序中最早的 8 个错误”，避免只挑最能讲故事的图片。

按类别统计召回率可以发现：总体准确率改善，可能主要来自少数容易类别。对图像做可视化时，务必使用与评估完全相同的噪声生成方式；不要展示一张“看起来差不多”的另一次扰动，然后声称它对应某个预测。

消融矩阵怎么扩展

想检查的解释	新实验	必须控制
增强强度是否关键	训练噪声 0、0.05、0.15、0.3	结构、轮数、种子、选择规则
干净验证是否偏好某类模型	比较不同验证选择目标	新协议；不根据当前测试结果选目标
固定轮数是否掩盖成本	固定耗时或更新数重做比较	同一设备、热身、计时范围
效果是否只属于当前任务	在另一图像任务验证	数据许可、相同扰动定义

这些是扩展建议，不能放进“已完成实验”表。若只能补一个，优先选最能区分替代解释的实验，而不是最可能得到高分的实验。

短论文逐段写法

引言：说明小模型可能面对输入噪声，而计算预算与干净性能仍重要，提出具体问题。不要写“深度学习发展迅速”后直接跳到方法。

方法与协议：说明三组结构、输入尺度、噪声裁剪、固定划分、5 轮训练、验证损失选择、种子和预算。测试噪声的单位是 `[0,1]` 像素尺度上的标准差。

结果：先回答主问题，再报告代价。写出不同噪声条件下的差异，给出每种子值或误差条，并说明只研究一个固定划分。

讨论：区分观察与推测。你可以说增强提升了本协议下的噪声准确率；若没有梯度、表示或额外机制实验，就不能断言“模型学到了某种特定不变性”。

检查点：读者能否仅凭你的论文与代码重建输入扰动？如果不知道噪声加在归一化前还是后、是否裁剪、评估时是否随机重采样，协议仍不完整。

PROJECT LAB

18 项目三：强化学习的问题设定

数据由行动产生

监督学习通常先拿到一个数据集；强化学习中，智能体行动影响下一状态、奖励和之后能见到的数据。因此，探索不是训练之外的小附件，而是学习过程的一部分。

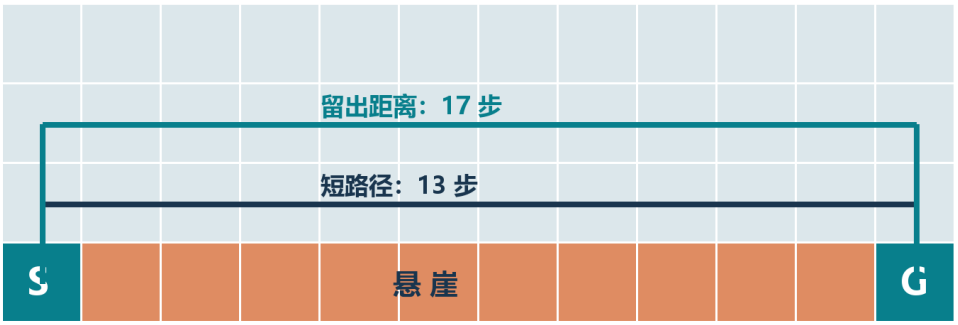
一个马尔可夫决策过程可描述为状态、动作、转移、奖励和折扣因子。策略 π(a|s) 决定在状态 s 采取动作 a 的概率。动作价值 Q(s,a) 描述从某状态动作出发、后续按某种规则行动时的期望折扣回报。[13]

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$$

γ 越小，越强调近期奖励。这个定义本身没有告诉你探索策略、训练预算和评价方式，它们必须单独说明。

悬崖行走环境

使用经典 4×12 网格设定：左下角起点，右下角终点，底行中间 10 格是悬崖。每一步奖励 -1；进入悬崖奖励 -100，并立即回到起点；只有到达终点才真正终止。这与把跌落直接当作 episode 结束的变体不同。[13]



跌落：-100，回到 S，回合继续；到 G：真正终止。

路径为概念示意，实际策略由各次训练的 Q 表决定。

图 18-1：悬崖网格与两类路径的示意

图中路径表示直观的短路径和留出距离的路径，不是本书所有种子都学到的实际策略。真实策略要从保存的 Q 表计算。

本书状态编码为 `row * 12 + col`，行从 0 到 3，列从 0 到 11。起点为 36，终点为 47。动作索引 0、1、2、3 分别表示上、右、下、左。越过边界时停留在边界格，仍消耗一步。

探索策略

采用固定 ε=0.1 的 epsilon-greedy 策略：90% 的时候选择当前 Q 值最大的动作，10% 的时候在全部四个动作中均匀随机选择。随机分支也可能恰好选到贪心动作，因此非贪心动作的总概率不一定恰好为 10%。

Q 值平局时随机打破平局。始终选索引最小的动作会引入方向偏好，尤其在初始化为零的网格环境里，可能显著改变早期轨迹。

研究问题

在固定探索率与训练回合数下，Q-learning 与 SARSA 的在线训练表现、冻结后的探索表现、最终贪心策略表现是否一致？我们的假设是它们可能给出不同排名，因为不同更新目标对探索风险的处理不同。

动手：手工画出一条沿悬崖上方的最短安全路径。起点上移一步，向右 11 步，下移一步，总共 13 步，因此无跌落时回报为 -13。再画一条上移两格的安全绕行路径，长度为 17，回报为 -17。

PROJECT LAB

19 项目三：实现 Q-learning 与 SARSA

两个算法只改一处，但含义不同

Q-learning 的目标使用下一状态的最大动作价值：

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right]$$

SARSA 的目标使用行为策略实际选出的下一动作：

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma Q(s',a') - Q(s,a)\right]$$

两者都可以通过 epsilon-greedy 行动。差别在于学习目标：Q-learning 使用贪心后续动作的价值，SARSA 使用当前行为策略后续动作的价值。这是 off-policy 与 on-policy 的一个具体入口。[13]

对于真正终止的下一状态，bootstrap 值为零。若只是为了节省计算达到步数上限而截断，我们把它看作外部截断，最后一次更新仍保留 bootstrap；随后新回合从起点开始。这个约定要与环境任务定义一起报告。

核心实现

PYTHON

```
next_action = None if terminated else action(q, next_state, eps, rng)
if terminated:
    bootstrap = 0.0
elif algorithm == "qlearning":
    bootstrap = q[next_state].max()
else:
    bootstrap = q[next_state, next_action]
target = reward + gamma * bootstrap
q[state, act] += alpha * (target - q[state, act])
```

新手常见错误包括：先选了下一动作，执行时又重新抽一次；把 SARSA 的实际下一动作替换成最大值；跌入悬崖时终止；真实终止后仍 bootstrap；评估时继续更新 Q。逐条检查比增加训练回合更重要。

冻结协议与运行

项目	设置
环境	4×12；跌落重置起点；抵达终点终止
初值与更新	Q 全零；学习率 0.5；折扣因子 1.0
行为策略	epsilon-greedy，epsilon 固定 0.1，平局随机
训练	每种子 500 回合；每回合最多 1,000 步
重复	10 个训练种子，0 至 9
冻结后评估	不更新 Q；分别用 epsilon=0 与 epsilon=0.1 各评估 100 回合

两种算法使用相同训练种子编号与环境定义，但由于动作和轨迹不同，后续消费的随机序列会分叉。不要声称它们经历了逐步完全相同的环境噪声。当前环境转移本身确定，主要随机性来自动作选择。

```bash
python p3_cliff.py --seeds 0 --episodes 20 --eval-episodes 10 --out runs/p3-check
python p3_cliff.py --out runs/p3-main
python analyze.py runs/p3-main/results.json --metric train_tail_return --pair sarsa qlearning
```

主比较按训练回合固定预算，同时保存实际转移步数。若研究“每次环境交互的样本效率”，应增加固定步数协议；相同回合数不等于相同交互预算。

三种评估不要混在一起

在线训练回报：边学习边探索，模型参数在变化。本书取最后 100 个训练回合的平均值，每种子得到一个统计量。

冻结后探索回报：Q 不再变化，但保留 epsilon=0.1，用来测最终学得策略面对同样探索机制时的表现。

冻结后贪心回报：Q 不再变化，epsilon=0，衡量部署时不主动探索的表现。平局仍随机。每种子评估 100 回合，只降低该策略评估的蒙特卡洛噪声，不把训练种子数从 10 变成 1,000。

PROJECT LAB

20 项目三：从失败中形成研究结论

本书实测

以下保留所有 10 个种子的结果，回报越高越好，即负值越接近零越好。均值后为跨训练种子的样本标准差。

指标	Q-learning	SARSA
训练尾段 100 回合平均回报	-46.78 ± 10.03	-26.20 ± 6.45
冻结后探索平均回报	-51.78 ± 7.63	-30.44 ± 16.34
冻结后贪心平均回报	-13.00 ± 0.00	-213.60 ± 414.47
冻结后探索每回合跌落数	0.35 ± 0.07	0.04 ± 0.03
每次训练实际转移步数	10441.20 ± 112.71	12939.70 ± 915.32

贪心评估成功率：Q-learning 所有种子均为 100%；SARSA 有 8 个种子为 100%，2 个种子为 0%。不能只报告成功种子的 -17 回报。

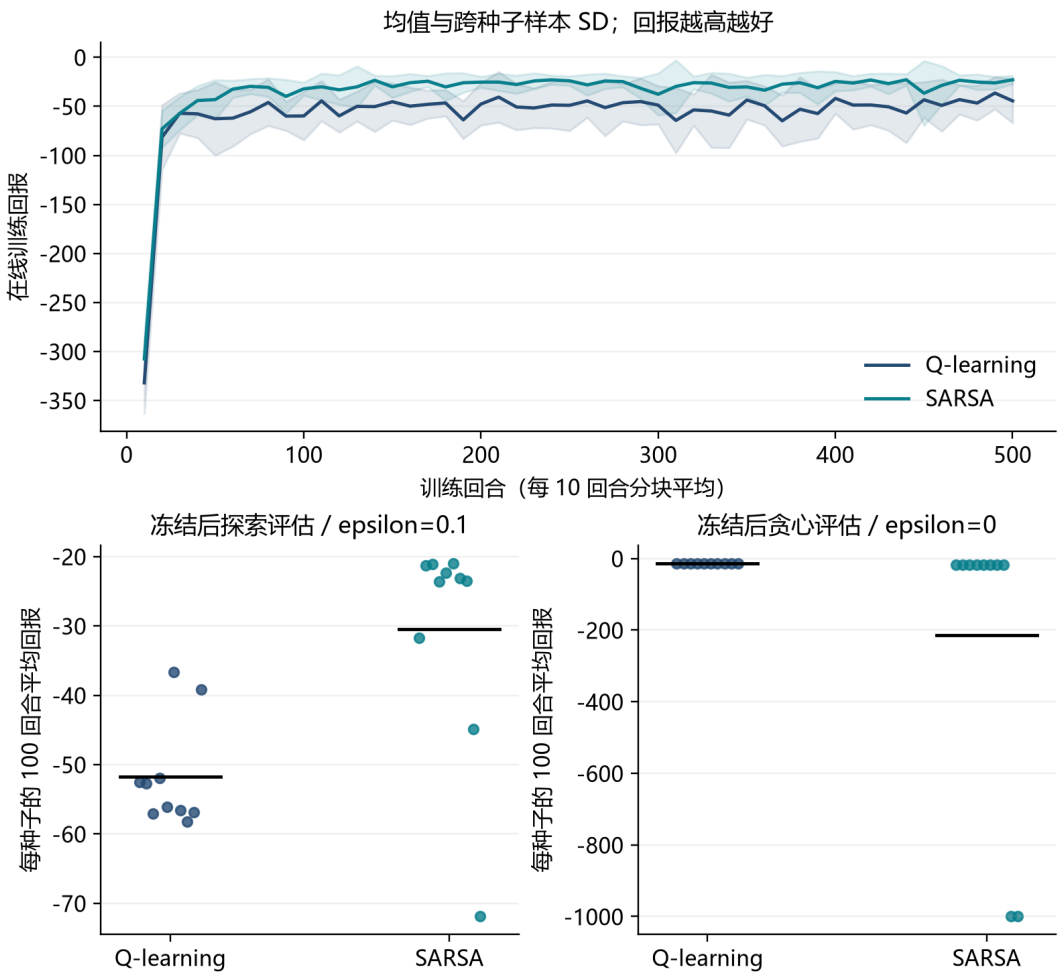


图 20-1：训练曲线与不同评估协议的种子结果

SARSA 在训练尾段与冻结后探索评估中平均更好、跌落更少。但冻结后贪心评估中，种子 8、9 的策略未能在 1,000 步内到达终点，回报为 -1,000；其余 8 个种子为 -17。Q-learning 的全部种子为 -13。

这个现象不能用“总体平均还行”抹掉。它说明当前训练预算与固定学习率下，SARSA 的最终贪心行为不稳定。也不能把它写成“SARSA 在理论上无法收敛”：本实验没有验证收敛理论所要求的全部条件，固定学习率和有限训练也不构成渐近结论。

诊断失败策略

先从 Q 表生成每个格子的最大值动作，标出平局、未访问和循环。对失败种子记录前 30 步的状态，检查是否出现两个格子之间来回移动。再比较失败状态的候选 Q 值是否非常接近，判断有限更新可能造成的策略不稳定。

本书保存的失败记录是观察；“因为哪些状态访问不足而失败”属于待验证解释。要支持解释，可以增加访问次数统计、画动作价值差值分布，或重新设置训练预算进行探索实验。不要跳过诊断直接删除种子。

图表怎样不误导

学习曲线可以用每 10 回合分块平均减少噪声，但必须注明窗口，保留原始 CSV。横轴是回合时，就不要称为交互步数。曲线阴影如果画标准差，要写“跨种子标准差”，不能写成“95% CI”。

最终贪心回报有极端失败值时，用每种子散点比只有一根柱子更诚实。可以补充成功率与中位数，但不能用新的统计量把失败隐藏掉。回报分布偏态时，读者尤其需要看到原始值。

从教学报告走向一个新问题

一个合理扩展是：在固定环境交互步数而非回合数时，探索率衰减如何影响两种算法的安全与最终贪心策略？你需要重新定义预算、增加 epsilon 日程、报告每种子成功率和转移步数，并用独立协议解释结果。

如果要进入深度强化学习，下一站可以是用神经网络近似 Q 值的 DQN，学习经验回放、目标网络与终止处理。先确保表格版环境和评估可信，再把表示换成网络。不要同时更换环境、网络、算法和评价方式，否则很难定位问题。

研究报告结构

题目可写“固定探索率下 Q-learning 与 SARSA 的训练及贪心评估差异”。贡献写成：在明确的教学设置中复现探索风险现象，并报告有限预算下的策略失败。结果章节按三种评估协议展开，讨论章节分析预算差异、失败种子与推广限制。

检查点：你的摘要是否保留了失败种子和预算边界？如果读摘要的人会误以为 SARSA 在所有评价方式都更好，就必须重写。

RESEARCH PRACTICE

21 统计与作图：让数字经得起追问

先明确你在平均什么

项目一每个种子重新生成数据，项目二各种子共享固定数据划分，项目三每个种子训练一个独立智能体。这三种均值的意义不同。统计分析的第一步是识别独立重复单位，而不是立刻选择显著性检验。

若对照组和方法组在相同种子、数据划分或任务条件下运行，可以先计算每对的差值，再分析差值。这样保留条件配对结构。若只有一组有某个种子，不要悄悄把两边不同样本直接配起来。

按种子做 bootstrap

先对相同种子的两组结果计算配对差值：

$$d_i = A_i - B_i.$$

下面演示重采样均值，完整实现见 `analyze.py`：

```python
rng = np.random.default_rng(2026)
resampled = rng.choice(differences, size=(10000, len(differences)), replace=True)
bootstrap_means = resampled.mean(axis=1)
interval = np.quantile(bootstrap_means, [0.025, 0.975])
```

它近似描述在当前抽样机制和样本条件下，差值均值估计的不确定性。区间不是“真实均值有 95% 概率落在这个已算区间里”的直接频率学解释，也不是跨任务的普遍保证。种子很少、分布有极端值或实验经过反复挑选时，区间尤其需要谨慎解释。

强化学习研究已经系统讨论少量运行与聚合指标的不确定性，建议把区间和分布一起报告，而非只比较点估计。[14] 本书只演示单任务、种子级 bootstrap；多任务评估需进一步考虑任务与种子的分层结构。

效应大小比一个标签更重要

准确率从 80% 到 82%，增加的是 **2 个百分点**；相对准确率增加为 2.5%；错误率从 20% 降到 18%，相对错误率下降 10%。三种表达都可能正确，但含义不同。写论文时选最直接的单位，并给出原始数值。

“差异不显著”不等于“两个方法等效”；也可能只是样本太少。想主张等效，需要事先定义有实际意义的容差并采用适当设计。初学阶段先报告差值、离散程度、预算和局限，比硬套一个检验标签更可靠。

图的语法

每张图应有一个明确问题。横轴是什么自变量？纵轴用什么单位？图例代表哪些配置？点是单个种子还是均值？误差条是什么？标题和图注应让读者离开正文也能理解。

配色用少量可区分颜色，再加实线、虚线、标记形状，不仅靠红绿区分。柱状图若表示大小应谨慎处理截断坐标；准确率曲线可以使用局部范围，但要清楚标轴。科学图不需要 3D 效果、渐变背景或装饰性阴影。

结果到结论的审计

检查三件事：每条结论是否对应图表；每张图表能否回溯到原始记录；每条原始记录能否回溯到代码、数据和协议。任何一处断开，都先修复证据链。

动手：为项目二的强噪声差值写一句带单位的话，再补一句范围限制。答案应包含“百分点”和“当前数据划分、噪声定义及训练预算”，而不只是“明显提升”。

RESEARCH PRACTICE

22 把证据组织成论文

从实验记录形成初稿

科研写作不是实验结束后的翻译工作。立项书是引言的原料，文献卡是相关工作的原料，协议是实验设置的原料，实验日志是讨论与局限的原料。平时不记录，最后就只能靠记忆填补关键细节。

推荐先写方法和实验设置，再放图表，再写结果与讨论，最后完成引言、摘要和标题。这个顺序让主张围绕现有证据生长。最终论文的阅读顺序仍按目标模板组织。

每个部分要回答什么

部分	核心问题	常见错误
标题	具体研究了什么、在什么范围？	用“通用”“革命性”超出证据
摘要	问题、方法、证据、结果与边界是什么？	只有动机，没有定量结果
引言	为什么值得问？现有认识缺什么？	宏大背景与具体实验脱节
相关工作	别人回答了什么？你与之如何关联？	只罗列论文，没有比较
方法	模型或系统怎样工作？	变量未定义，关键操作省略
实验	结果是怎样测出来的？	测试集调参、预算不透明
讨论与局限	还有哪些解释和未验证条件？	只写“未来扩大规模”
结论	本文实际新增了哪条认识？	重复广告式贡献

一份完成态的项目一摘要

本文研究异尺度输入下，标准化对有限步逻辑回归优化的影响。我们生成 2,000 条 12 维合成样本，在相同划分、400 次梯度更新和四个学习率候选下，比较原始输入与训练集标准化。各组按验证交叉熵选择配置，并在五个数据种子上评估。原始组与标准化组的测试准确率分别为 51.60% 和 86.75%，跨种子样本标准差分别为 2.48 和 2.06 个百分点。结果支持标准化改善当前有限预算优化的判断；它不说明充分优化后的统计上限，也不代表所有真实数据分布。

这是一段基于本书实测的教学摘要。它写出了研究条件、选择规则、重复单位、数字和边界。因为研究设计来自教学目的，不能把它直接包装成未经检索的新学术贡献。

结果段落的三个层次

先给观察：“在噪声标准差 0.3 下，增强 CNN 的平均测试准确率高于基础 CNN。”再给证据：均值、种子数、离散程度、差值和对应图表。最后给解释与限制：“与训练时接触噪声的设定一致，但尚未证明对其他扰动或数据集有效。”

避免把图表数字逐个重读一遍。正文应解释最重要的比较、异常与代价。每个段落都围绕一个问题，不要把结构、硬件、结果和未来工作堆进同一句话。

Markdown 到论文排版

Markdown 很适合草稿：`#` 形成层级，表格组织比较，反引号标识代码，引用用统一编号与文献列表。图片使用相对路径，图注给出编号、含义与来源。块级公式使用两个分别独占一行的 `$$` 分隔符，公式块前后各留一个空行；不要把公式放进普通代码块，也不要把分隔符直接拼在中文段落中。本书的独立数学公式统一采用这一写法，需要编辑器支持数学公式渲染。

正式投稿再迁移到目标 LaTeX 或 Word 模板。迁移时逐项核对公式编号、图表引用、参考文献和字体嵌入。不要为了排版先删掉局限，也不要为了凑页数扩大没有证据的主张。

检查点：让一位未参与实验的同学只读摘要和主图，复述你的主结论。若复述的范围比你实际验证的大，应收紧表述。

RESEARCH PRACTICE

23 复现包、同行反馈与投稿

交给别人之前先自己复现一次

从一个干净目录开始，按 README 安装依赖，运行一个小实验，再运行完整协议并重算统计。目标不是追求每台机器每位小数完全相同，而是让别人能理解合理差异来自哪里。

研究包至少包括：README、协议、源代码、依赖说明、数据获取方式、种子清单、原始结果、汇总脚本、图表生成说明和论文。无法公开的数据或资源要说明访问条件；不要发布你没有权利分发的内容。

MARKDOWN

```
## 如何复现主表
1. 使用 Python 3.12 创建环境。
2. 安装 requirements-core.txt。
3. 运行书中指定的最终评估命令。
4. 使用 analyze.py 生成按种子的汇总。
5. 核对种子、样本数和选择规则，而非只看一格分数。
```

寻求同行反馈

把请求缩小到一个可回答问题：“这张图能支持摘要第二句吗？”“这个数据划分与部署情境一致吗？”“复现时哪一步最先失败？”收到反馈后，记录问题、证据、处理方式和是否影响结论。

若复现者得出不同结果，先核对版本、默认参数、数据校验值、划分、设备与评估脚本。不要先把差异归因于对方“没按说明做”；说明不足也是研究包的问题。

投稿前核对目标场所

会议、期刊、研讨会和校内研究展示的定位不同。选场所时阅读当年的官方征稿和作者指南，核对主题、截止时间、页数、匿名要求、伦理与材料政策。具体规则会变化，本书不提供一份永久有效的清单。

本书建议首先把结果用于课程研究报告、读书会或本科生科研训练。在导师指导下判断创新性与证据是否达到目标要求。投稿只是交流的一种形式，研究包可复现、结论可信同样是实质成果。

学术诚信与 AI 辅助

不得编造运行、筛掉不利种子而不说明、伪造引用或把别人代码改名当作自己的方法。引用开源代码与数据，检查其许可；作者署名和贡献与合作者提前沟通。

可以用 AI 解释错误、辅助草稿或生成待检查代码，但你要能解释每一条重要逻辑，核对引用原文，并遵循学校与目标场所对 AI 使用的当期规定。把未公开数据、凭据或合作者材料交给外部工具前，先确认使用授权。

检查点：你能否对论文中每个主要数字说明“谁运行、怎样运行、记录在哪、怎样重算”？

RESEARCH PRACTICE

24 继续走向独立研究

不用一次学完所有技术

下一阶段按研究问题补能力：做视觉就深入卷积、注意力、数据增强与迁移；做强化学习就深入函数逼近、DQN、策略梯度和离线评估；做系统就学习性能分析、并发、网络和可重复基准；做检索或语言任务则学习表示、索引、数据构建与评价。

无论方向如何变，数据来源、基线、公平预算、统计单位、错误分析和证据边界都不会消失。你在小项目中练出的习惯，可以迁移到更大的服务器与模型。

一张能力验收表

能力	你应该能独立完成的动作
编程	将一段数据处理写成输入输出明确的函数
环境	确认解释器，建立独立环境，记录依赖
工具	用 Git 关联代码，用终端与服务器运行任务
文献	提取主张、协议与局限，检查引用支持关系
设计	在运行前写好数据划分、选择规则与预算
验证	用小样本和已知答案检查关键逻辑
分析	保存全种子结果，解释异常和失败
写作	用图表支撑主张，给出可复现方法与边界

不要用“读过多少论文”或“跑过多大模型”作为唯一进度条。更有价值的指标是：问题是否更清楚，实验是否更可信，别人是否更容易理解并验证你的工作。

下一次立项的起点

把感兴趣的问题改写成一页立项书，包含一个最小实验、一个可信基线、一个会推翻预期的结果和一个明确的计算预算。新的知识会在具体问题中逐步长出来。

RESEARCH PRACTICE

附录 A 可直接使用的研究模板

实验日志

MARKDOWN

```
# YYYY-MM-DD 实验日志
## 本次实验想回答的问题
## 运行身份
- run 目录、commit、dirty 状态：
- 命令、环境锁文件、设备：
## 运行前预期
## 实际观察
- 原始记录路径：
- 数值、曲线、错误：
## 解释与替代解释
## 协议变更
- 改了什么、为何改、是否已看测试结果：
## 尚待解决的问题与候选实验
```

一次实验的检查清单

□ 研究问题有明确条件、比较对象与指标。

□ 训练、验证、测试的职责与划分单位清楚。

□ 数据处理只在允许的数据上拟合。

□ 候选数量、选择规则、预算和种子提前声明。

□ 小样本与已知答案检查通过。

□ 每次运行有独立目录，失败记录保留。

□ 每个种子的结果都进入汇总。

□ 图表标明单位、重复单位与误差含义。

□ 结论没有超出任务、数据与计算预算。

□ 同学可以根据 README 重算主表。

结果表模板

方法	种子数	数据与预算	主指标均值	样本标准差	训练成本	失败数
基线	待填	待填	待运行	待运行	待测量	待核验
方案	待填	同协议	待运行	待运行	待测量	待核验

这是空白模板，待填不代表已测得。不要为了让表格完整而补猜测数字。未运行、运行失败、指标不适用是不同状态，应分别标注。

组会汇报的五页结构

1. 问题：上次到这次，具体减少了什么不确定性。
2. 协议：数据、基线、指标和预算有什么变化。
3. 证据：最重要的一张图与一张表。
4. 异常：失败种子、替代解释和当前阻塞。
5. 决策：下一步做哪一个实验，为什么。

RESEARCH PRACTICE

附录 B 排错速查与练习答案

排错路径

症状	可能原因	首个定位动作
损失 NaN	输入或梯度异常、学习率过大	检查有限值，打印输入范围与损失
准确率异常高	泄漏、重复样本、标签错误	查划分与处理参数来源
训练好验证差	过拟合、分布差异、处理不一致	比较数据分布和变换流程
GPU 利用率低	数据、CPU 或小批量成为瓶颈	分别计时读取、前向、反向
显存耗尽	批太大、保留计算图、缓存输出	减批量，检查列表中是否存张量图
远程断连后找不到任务	会话或调度使用方式不当	查看调度状态和日志，不盲目重提
RL 回报上不去	奖励、终止、bootstrap 或策略逻辑错	手算并打印 10 条转移
两次结果不同	环境、数据、随机源或并行差异	对照元数据，缩到 CPU 单线程调试

概念题与答案提示

题 1：为什么不先对全部数据标准化再划分？ 因为均值和标准差会使用未来评估数据的信息，使评估过程与真实“只见训练数据”的场景不同。应先划分，再用训练集拟合变换。

题 2：验证集能反复看吗？ 它本来用于开发和选择，但看得越多、搜索越广，也可能逐渐对它过拟合。需要记录搜索预算；重要结论仍要在协议冻结后做未见评估。

题 3：5 个模型各评估 100 个 episode，是 500 个独立训练样本吗？ 不是。训练重复单位仍是 5 个；每个模型的 100 次评估用于估计该策略表现。

题 4：项目三跌落时为什么不把 bootstrap 设为 0？ 因为本环境跌落后立即回到起点，episode 没有终止。后续奖励仍属于同一回合的回报。

题 5：固定轮数比较为何还要报告时间？ 相同轮数不保证计算量相同，报告时间能帮助读者理解代价；时间还要说明设备和计时范围。

题 6：方法均值更高，能否说明所有种子都更好？ 不能。需要查看配对差值和分布；平均优势可以与个别失败同时存在。

题 7：项目二训练噪声改善强噪声准确率，能否称“解决图像鲁棒性”？ 不能。它只验证特定数据、像素噪声、裁剪、样本量和预算，其他扰动仍未知。

三个项目的评分建议

研究问题与协议 20 分；实现与正确性检查 25 分；原始记录与可复现性 20 分；分析与局限 20 分；写作图表 15 分。高分不要求你的新方案一定获胜，但要求没有把失败隐藏起来。

RESEARCH PRACTICE

附录 C 配套包与本次运行说明

文件清单

文件或目录	用途
p1_scaling.py	项目一：生成合成数据、调参、最终评估
p2_fashion.py	项目二：读取 IDX、训练三组模型、噪声评估
p3_cliff.py	项目三：环境、训练、两种冻结评估
common.py	新建输出目录、保存元数据与结果
analyze.py	均值、样本标准差、配对 bootstrap
verify.py	划分、环境转移、统计和模型形状检查
make_figures.py	从本次记录重绘书中实验结果图
templates/	立项、实验日志和论文草稿模板
worked_reports/	三个项目的精简成稿示范，含实测结果与局限
reference_runs/	本书主实验的逐种子记录与小型预测文件
VALIDATION.md	实际执行的验证与环境说明

数据集与 PyTorch 安装包不包含在配套包内，首次运行项目二需要下载官方数据。为了控制附件大小，参考运行不附模型检查点；训练脚本会为你自己的运行保存检查点。项目三的小型 Q 表保留，便于检查失败策略。

三份成稿示范展示如何把问题、方法、设置、结果和讨论组织成连贯短文。它们是教学范例，不是已发表论文，也不应未经独立研究就作为自己的原创论文提交。

真实运行与示例的分界

第 14、17、20 章的数字和实验图来自本次主实验记录。流程图、模型形状图、悬崖路径图是教学示意。所有“扩展方向”均是尚未完成的建议。`--smoke` 只验证程序贯通；表格模板中的“待运行”不是结果。

本次没有在学校服务器、Slurm 集群或 CUDA 设备上执行训练；相关命令是依据官方文档整理的迁移模板。没有训练 DQN，也没有完成任何投稿或声称学术录用。

本次运行环境

项目	实际环境与重复
通用	Windows 11；Python 3.12.14；NumPy 2.3.5；CPU
项目一	5 个数据种子；每组 400 步；四候选学习率
项目二	PyTorch 2.14.1+cpu；CPU 两线程；3 组 × 5 种子 × 5 轮
项目三	2 算法 × 10 种子 × 500 回合；冻结后各 100 回合评估

训练计时仅覆盖训练循环及每轮验证，排除数据下载、模型构造与最终测试；本次未获得可核对的 CPU 型号，因此不把耗时用作跨硬件性能结论。

配套包中保留每个源文件的 SHA-256 摘要，并提供逐种子结果。这个教学工作目录没有可用的研究仓库 commit，因此元数据如实记录为 `unavailable`，没有伪造提交号。你自己的研究应按第 06 章建立 Git 版本记录。

版本与再现边界

本书记录了本次实际使用的依赖版本，不将它们称为永久“最新版本”。CPU 与 GPU、不同操作系统和依赖版本可能产生数值差异。先验证样本数、划分、选择规则和趋势；若关键结论改变，应调查并报告。

RESEARCH PRACTICE

附录 D 首次实验：环境准备与运行

准备工具

如果电脑没有 Python，先到 Python 官方网站 获取适用于你的操作系统的 Python 3.12 安装方式，具体 Windows 安装行为参考 官方 Windows 指南。安装后重新打开 PowerShell，输入 `python --version`。若系统已有多个 Python，使用 `py -3.12 --version` 检查指定版本，并用 `py -3.12 -m venv .venv` 创建环境。

若输入 `python` 打开了应用商店，说明命令目前没有指向你需要的解释器；先按官方指南确认安装与命令路径。不要把反复安装库当作解决解释器路径问题的方法。

编辑器只要能保存 UTF-8 文本与 `.py` 文件即可。Git 从 官方页面 按系统安装；重新打开终端后运行 `git --version`。第一个实验可以先完成，再按第 06 章把代码和记录纳入版本管理。

认识一段小程序

建立一个 `hello_research.py` 文件，写入以下内容，然后在所在目录执行 `python hello_research.py`：

```python
scores = [0.80, 0.82, 0.78]

def mean(values):
    return sum(values) / len(values)

for index, score in enumerate(scores):
    print(index, score)

print("mean:", mean(scores))
```

列表保存多个值，函数给一段计算命名，`for` 循环逐个处理元素，缩进表达所属代码块。最后一行应输出约 0.8。如果程序没有运行，先检查是否误存成 `.py.txt`，以及终端当前目录是否正确。

完成第一个实验

解压本书配套代码，进入含有 `p1_scaling.py` 的目录。按第 03 章创建环境并安装 `requirements-core.txt`，运行 `verify.py`。看到 `PASS` 后，运行项目一的单种子开发命令。打开生成的 `results.json`，找到两组的 `val_loss`，用自己的话解释它们分别代表什么。

这时不要急着加 `--finalize`。先读第 10、12、13 章，理解为什么有验证集，为什么按规则选择模型，为什么最终测试需要协议冻结。

保存研究记录

复制 `templates/log.md`，写下命令、解释器路径、输出目录、预期和观察。把文件交给他人，请他指出哪里还不清楚。这份记录应当足以说明实验做了什么，并支持重新运行。

检查点：不用翻终端历史，你仍能在研究日志里找到怎样重做这次实验。如果做不到，先完善日志，不要用新的运行覆盖旧记录。

RESEARCH PRACTICE

参考资料与继续阅读

本书原创叙述、练习与项目围绕教学目的编写。下面列出核对过的官方文档、原始论文和作者资源；访问日期为 2026-10-18。技术细节应结合所安装版本阅读。

- **[1]** Python 文档，Virtual Environments and Packages。官方教程。用于环境隔离与解释器管理。
- **[2]** PyTorch，Start Locally。官方安装选择器。用于选择平台和计算后端。
- **[3]** MIT，The Missing Semester of Your CS Education。课程主页。用于补命令行、开发工具和调试。
- **[4]** Scott Chacon、Ben Straub，Pro Git, 第二版。官方在线书。用于版本控制概念与操作。
- **[5]** OpenBSD，ssh(1)。OpenSSH 手册。用于远程登录和本地端口转发。
- **[6]** SchedMD，sbatch 与用户指南。sbatch 手册、入门指南。用于作业提交与队列概念。
- **[7]** PyTorch，Reproducibility。本次核对的 2.14 文档。用于随机源和跨平台再现边界。
- **[8]** S. Keshav，How to Read a Paper, ACM SIGCOMM Computer Communication Review, 37(3), 2007。大学托管原文。用于三轮阅读法。
- **[9]** scikit-learn，Common Pitfalls and Recommended Practices。官方文档。用于数据泄漏与预处理拟合范围。
- **[10]** Ian Goodfellow、Yoshua Bengio、Aaron Courville，Deep Learning, MIT Press, 2016。作者网站。用于系统学习神经网络与优化。
- **[11]** Han Xiao、Kashif Rasul、Roland Vollgraf，Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms, 2017。官方数据仓库。用于数据来源、格式和许可信息。
- **[12]** PyTorch，Learn the Basics。官方入门教程。用于张量、训练、评估与模型保存。
- **[13]** Richard S. Sutton、Andrew G. Barto，Reinforcement Learning: An Introduction, 第二版，MIT Press, 2018。作者资源、出版社页面。第 6 章介绍 SARSA、Q-learning 与悬崖行走。
- **[14]** Rishabh Agarwal、Max Schwarzer、Pablo Samuel Castro、Aaron Courville、Marc G. Bellemare，Deep Reinforcement Learning at the Edge of the Statistical Precipice, NeurIPS 2021。原始论文、作者项目。用于理解少量训练重复下的评价不确定性。